Projet Robotique Infotronique Mécatronique

Iwen JOMAA, Baptiste GEFFRAY
Université de Bordeaux

16 janvier 2026

Table des matières

1 Introduction **2**

2 Déplacements du bras **3**
2.1 Algorithme 3
2.2 Limitations 5

3 Déplacements de la voiture **5**
3.1 Gestion de la vitesse 5
3.2 Splines 5
3.2.1 Algorithme de base 5
3.2.2 Améliorations 6
3.2.3 Limitations 7
3.3 Rapidly-exploring Random Tree (RRT) 7
3.3.1 Algorithme 7
3.3.2 Limitations 9

4 Conclusion **9**

1 Introduction

Ce projet consiste à simuler dans **Unity** un scénario de robotique mobile (cf. Fig. 1) comprenant :

— une **plateforme mobile** (voiture) se déplaçant sur un terrain, avec une trajectoire générée à partir de waypoints,
— un **bras articulé** monté sur la voiture, piloté en **cinématique inverse** pour atteindre une position cible,
— une **mission** complète de type “aller saisir un objet, revenir, déposer”.

FIGURE 1 – Vue générale de la scène Unity : plateforme mobile équipée d'un bras articulé tenant l'objet cible.

L'orchestration globale est réalisée par `MissionHandler` qui enchaîne des actions via des callbacks et des coroutines. La navigation de la voiture est gérée par `CarController` (suivi de spline ou planification RRT), et la manipulation par `ArmController`.

Chaînage de la mission

Le déroulement implémenté est le suivant :

1. déplacement de la voiture vers la position cible (en conservant une distance `distanceToKeep` de l'objectif),
2. déplacement du bras vers la cible et tentative de saisie,
3. remontée du bras en position par défaut,
4. retour de la voiture à l'origine,
5. bras en position de dépose,
6. relâchement de l'objet,

7. retour du bras en position de base.

Listing 1 – Extrait : enchaînement par callbacks (MissionHandler)

```
// Step 1 : moving car to target position
this.MoveCar(targetPosition, targetRotation, waypoints, this.distanceToKeep,
    handToTarget, () => this.OnError(true, null));

// Step 2 : moving arm to target position + grab
this.MoveArmWorldReference(targetPosition, true, true, putArmUp, () => this.
    OnError(false, putArmUp));
```

2 Déplacements du bras

2.1 Algorithme

Le contrôle du bras est assuré par `ArmController`. Le modèle correspond à un bras à deux segments :

— haut du bras (épaule → coude),

— avant-bras (coude → main).

Les longueurs sont calculées au démarrage à partir des `Transform` qui composent le bras.

Changement de repère Pour rendre le calcul indépendant de l'orientation du véhicule, la cible est convertie dans un repère "épaule" :

$$\mathbf{p}_s = R^{-1}(\mathbf{p}_w - \mathbf{p}_{\text{épaule}}), \qquad \mathbf{p}_w = \mathbf{p}_{\text{épaule}} + R\,\mathbf{p}_s,$$

où R est la rotation du véhicule (dans le code : `car.rotation`).

Cinématique inverse géométrique Soit $\mathbf{p}_s = (x, y, z)$ la cible dans le repère de l'épaule et $L = \|\mathbf{p}_s\|$. L'accessibilité est testée par :

$$L \leq L_1 + L_2.$$

Les angles sont ensuite calculés par une méthode géométrique (loi des cosinus) où l'axe y correspond à l'élévation verticale dans le repère de l'épaule :

$$\varphi = \operatorname{atan2}(x, z), \quad \alpha = \arccos\left(\frac{y}{L}\right)$$

$$\theta_1 = \alpha - \arccos\left(\frac{L_1^2 + L^2 - L_2^2}{2L_1L}\right), \qquad \theta_2 = \pi - \arccos\left(\frac{L_1^2 + L_2^2 - L^2}{2L_1L_2}\right).$$

Dans l'implémentation, l'épaule reçoit une rotation cible (pitch θ_1 et yaw φ) et le coude une rotation cible (pitch θ_2). Le mouvement est interpolé avec `Quaternion.RotateTowards` à la vitesse `rotationSpeed`.

Listing 2 – Pseudo-code : cinématique inverse géométrique (ArmController)

```
Algorithm ArmIK
Inputs: targetWorldPosition pw, shoulderWorldPosition ps0, carRotation R,
    lengths L1, L2
```

```
Outputs: (theta1, theta2, phi) or Failure
Begin
        // World -> shoulder reference frame
        ps <- Inverse(R) * (pw - ps0)
        L <- Norm(ps)
        If L > L1 + L2 Then Return Failure

        phi <- atan2(ps.x, ps.z)
        alpha <- arccos(ps.y / L)
        theta1 <- alpha - arccos((L1^2 + L^2 - L2^2) / (2*L1*L))
        theta2 <- PI - arccos((L1^2 + L2^2 - L^2) / (2*L1*L2))

        Return (theta1, theta2, phi)
End
```

Saisir et déposer La saisie est déclenchée par un test de proximité autour de la main dans un rayon `grabRange` calculé automatiquement à partir du collider de la main. Lorsqu'un rigidbody (hors voiture) est trouvé, l'objet est figé (vitesses à zéro, gravité désactivée), les collisions avec le véhicule sont ignorées, et l'objet est attaché à la main (cf. listing 3).

Listing 3 – Pseudo-code : saisir (ArmController)

```
Algorithm Grab
Begin
        If isHolding = false And grabEnabled = true Then
                For Each collider in OverlapSphere(handPosition, grabRange) Do
                        If collider belongsTo car Then Continue
                        If collider hasRigidBody Then
                                FreezeRigidBody(collider.rigidBody)
                                IgnoreCollisionsWithCar(collider)
                                AttachToHandPreservingWorldScale(collider)
                                isHolding <- true
                                Break
                        EndIf
                EndFor
        EndIf
End
```

Le dépôt effectue l'opération inverse : les collisions avec la voiture sont restaurées, l'objet est détaché de la main, et son rigidbody est débloqué (cf. listing 4).

Listing 4 – Pseudo-code : déposer (ArmController)

```
Algorithm Release
Begin
        If isHolding = true Then
                RestoreCollisionsWithCar(heldObject)
                DetachFromHand(heldObject)
                UnfreezeRigidBody(heldObject)
                isHolding <- false
        EndIf
End
```

2.2 Limitations

- **Modèle cinématique simplifié** : 2 articulations (épaule/coude) sans contraintes mécaniques ni modélisation détaillée (pas de limites articulaires explicites).
- **Pas d'évitement d'obstacles pour le bras** : la trajectoire articulaire n'est pas validée contre l'environnement (risque de collisions).
- **Saisie opportuniste** : le premier rigidbody détecté dans la sphère peut être saisi, ce qui peut provoquer un mauvais choix en cas de plusieurs objets.

3 Déplacements de la voiture

3.1 Gestion de la vitesse

Trois lois de vitesse existent : constante, accélération puis arrêt, et profil **trapézoïdal**. Le profil trapézoïdal est pré-calculé sur la longueur de la spline 3D :

- phase d'accélération ($v(t) = at$),
- phase à vitesse constante ($v(t) = v_{\max}$) si la distance le permet,
- phase de décélération symétrique ($v(t) = a\,(T - t)$).

Si la trajectoire est trop courte, le profil devient triangulaire (pas de plateau), avec $t_a = \sqrt{D/a}$.

3.2 Splines

3.2.1 Algorithme de base

Le mode **Spline** construit une trajectoire à partir d'une liste de points : position actuelle → waypoints → cible. La spline est construite en 2D (XZ), puis projetée sur la hauteur du terrain pour obtenir une spline 3D (cf. Fig. 2). Les derniers points trop proches de la cible sont retirés afin de respecter une distance d'arrêt `distanceToKeep`. Pendant le mouvement, la voiture progresse le long de la spline en intégrant une distance parcourue et en alignant le yaw sur la tangente locale (cf. listing 5).

Listing 5 – Pseudo-code : génération et suivi spline (CarController + SplineCreator)

```
Algorithm PlanAndFollowSpline
Inputs: startPosition, waypoints, targetPosition, distanceToKeep
Begin
        points <- [startPosition] + waypoints + [targetPosition]

        spline2D <- BuildSpline2D(points) // knots in XZ, y=0
        spline2D <- SmoothTangents(spline2D) // smoothing tangents and optional
            smoothing knots
        If checkValidity Then
                While spline2D invalid And attempts < maxRepathAttempts Do
                        (invalidPoint, previousPoint) <- FirstInvalidSegment(
                            spline2D)
                        bypass <- GenerateBypass(previousPoint, targetPosition,
                            invalidPoint)
                        Insert(bypass into points near previousPoint)
                        spline2D <- BuildSpline2D(points)
                        spline2D <- SmoothTangents(spline2D)
```

```
                EndWhile
        EndIf

        spline3D <- ProjectOnTerrainAndTrim(spline2D, distanceToKeep)
        SetupSpeedProfile(spline3D)

        traveled <- 0
        While not finished Do
                v <- SpeedAtTime(t)
                traveled <- traveled + v * dt
                u <- Clamp(traveled / Length(spline3D), 0, 1)
                position <- EvaluatePosition(spline3D, u)
                tangent <- EvaluateTangent(spline3D, u)
                MoveCarXZ(position)
                AlignYawTo(tangent)
        EndWhile
End
```

3.2.2 Améliorations

Lissage des tangentes La classe `SplineCreator` fournit `SmoothSplineTangents` afin de générer des tangentes plus "conduisibles". Le principe est :

- calculer une direction tangentielle aux nœuds à partir de la direction du segment entrant et du segment sortant,
- si la somme des deux directions est quasi nulle (demi-tour), choisir une direction perpendiculaire "stable". Pour cela on utilise la perpendiculaire qui est opposée à la sortie du nœud précédent pour éviter les formes en S,
- forcer les tangentes du premier et du dernier nœud à correspondre à des directions souhaitées (ici, la direction avant au démarrage du véhicule),
- optionnellement insérer un nœud de smoothing proche du début/fin si l'angle d'arrivée/départ est trop important (seuil `maxArrivingAngle`).

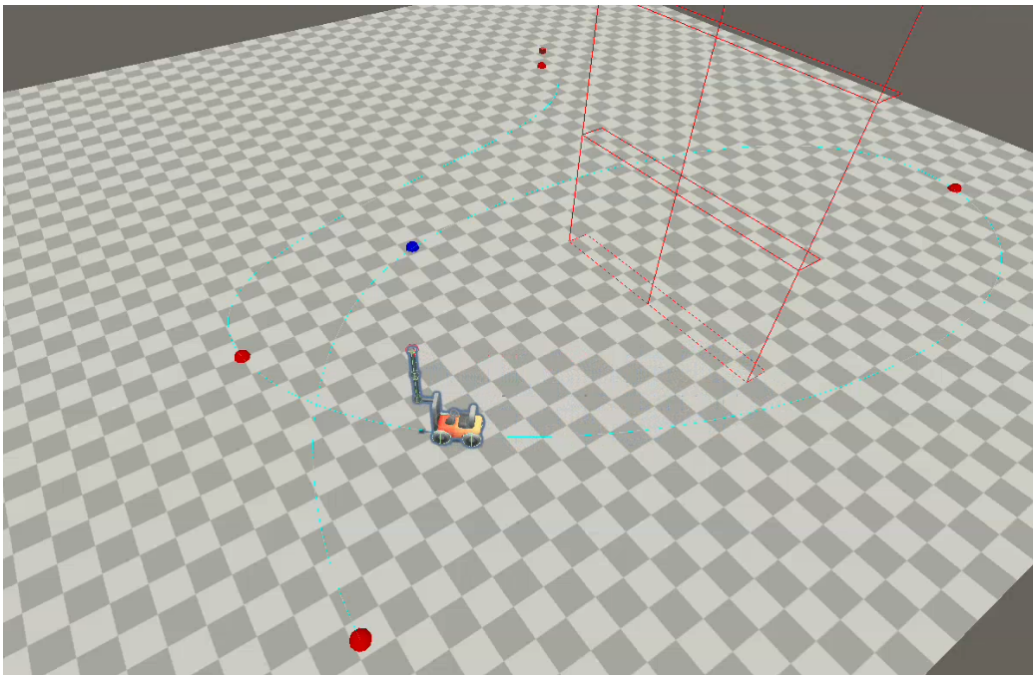

FIGURE 2 – Exemple de trajectoire générée à partir d'une spline classique, lissée pour un suivi fluide par la voiture.

3.2.3 Limitations

- **Projection terrain obligatoire** : la génération 3D utilise `Terrain.activeTerrain`. En l'absence de terrain actif, l'algorithme n'est pas défini.
- **Validation par discrétisation** : la validité (zones interdites, pente) dépend du pas d'échantillonnage, une augmentation de la précision entraîne donc une augmentation du coût de calcul.
- **Correction de chemin heuristique** : en cas d'échec, un point de contournement est proposé par offsets latéraux/avant et inséré dans la liste de points. Cette stratégie peut échouer pour des obstacles complexes.
- **Dynamique simplifiée** : la position est imposée sur x, z sans modélisation d'adhérence, de glissement ou de contraintes de braquage.

3.3 Rapidly-exploring Random Tree (RRT)

3.3.1 Algorithme

Algorithme de base Le planificateur **RRT** est implémenté dans `RRTPlanner`. Il construit un arbre en 2D (XZ) dont chaque nœud est un `TreeNode` (position + parent). Les bornes d'échantillonnage proviennent du terrain Unity (emprise x, z), ou de limites par défaut si aucun terrain n'est trouvé.

Le planificateur **RRT** échantillonne des points aléatoires avec un biais vers l'objectif sélectionnant de manière aléatoire directement l'objectif (cf. listing 6), cherche à chaque fois le nœud de l'arbre le plus proche, et étend l'arbre d'un pas fixe `stepSize` dans la direction de l'échantillon. Chaque nouveau segment est validé (zones interdites, pente, marge de largeur). Lorsque l'extension arrive suffisamment près de la cible, on tente la connexion finale et on reconstruit le chemin en remontant les parents (cf. Fig. 3).

Listing 6 – Pseudo-code : RRT 2D avec validation (RRTPlanner + PathValidator)

```
Algorithm RRTPath
Inputs: start, goal, stepSize, maxIterations, goalBias
Outputs: path or Failure
Begin
        tree <- { Node(start, parent = null) }
        For k = 1 to maxIterations Do
                If Random() < goalBias Then pos_rand <- goal
                Else pos_rand <- SampleUniformWithinTerrainBounds()

                pos_near <- NearestNode(tree, pos_rand)
                pos_new <- Steer(pos_near, pos_rand, stepSize)
                pos_new <- ClampToBounds(pos_new)

                If IsMoveValid(pos_near, pos_new) Then
                        Append(tree, Node(pos_new, parent = pos_near))
                        If Distance(pos_new, goal) < stepSize And IsMoveValid(
                            pos_new, goal) Then
                                Return BacktrackPath(Node(goal, parent = Node(
                                    pos_new)))
                        EndIf
                EndIf
        EndFor
        Return Failure
End
```

La méthode `FindFullRRTPath` enchaîne ce calcul entre les points successifs (waypoints inclus) et concatène les segments pour obtenir un chemin complet.

Simplification Une simplification gloutonne est proposée : depuis un index courant, on cherche le point le plus éloigné atteignable directement (segment valide) et on saute tous les points intermédiaires afin de réduire le nombre de segments (cf. listing 7).

Listing 7 – Pseudo-code : simplification gloutonne du chemin

```
Algorithm SimplifyPathGreedy
Inputs: pathPoints
Outputs: simplified
Begin
        simplified <- [pathPoints[0]]
        i <- 0
        While i < lastIndex(pathPoints) Do
                farthest <- -1
                j <- lastIndex(pathPoints)
                While j > i And IsMoveValid(pathPoints[i], pathPoints[j]) = false
                    Do
                        farthest <- j
                        j <- j - 1
                EndWhile
                If farthest = -1 Then Return pathPoints // cannot simplify
                Append(simplified, pathPoints[j])
                i <- farthest
```

```
        EndWhile
        Return simplified
End
```

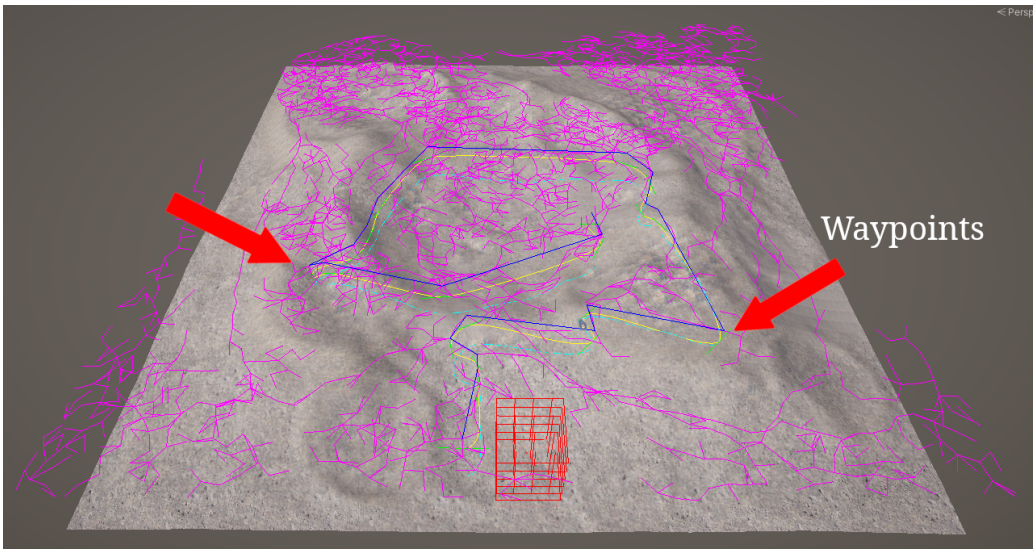

FIGURE 3 – Exemple de trajectoire générée par le planificateur RRT, simplifié, puis converti en spline lissée pour le suivi par la voiture.

3.3.2 Limitations

- **Pas d'optimalité** : l'algorithme RRT vise la faisabilité, pas le meilleur chemin.
- **Aléatoire** : la réussite et la qualité du chemin dépendent des paramètres (`stepSize`, `maxIterations`, `goalBias`) et du tirage.
- **Validation coûteuse** : chaque extension nécessite des tests (zones + pente + marge), ce qui peut être lourd en itérations.
- **Adaptation en spline** : le chemin RRT est une suite de segments droits, nécessitant une conversion en spline pour un suivi fluide par la voiture. Le processus peut générer des trajectoires qui ne correspondent plus exactement aux contraintes initiales (zones interdites, pente).

  Solution : Pour éviter ce problème, une fois la spline générée, on peut la faire passer par le même validateur de chemin que pour la spline classique avec son système de réajustement du chemin. Cette méthode fonctionne, mais la fonction de réajustement du chemin n'est pas suffisamment robuste pour garantir la validité de la trajectoire finale dans tous les cas.

4 Conclusion

Ce système met en place une chaîne complète comprenant navigation et manipulation. Le bras utilise une cinématique inverse géométrique et saisie et dépose les objets via la physique de Unity. La voiture suit des trajectoires continues grâce aux splines (lissées) et à un profil de vitesse trapézoïdal et une alternative de planification est fournie via RRT avec validation (zones d'évitement et pentes). Les limites principales proviennent de

l'absence de vérification des contraintes lors des mouvements du bras, ainsi que du caractère heuristique des stratégies de contournement. Des améliorations possibles incluent donc l'ajout de contraintes articulaires, la prise en compte des collisions du bras, et des stratégies de réajustement du chemin plus robustes appliquées aux splines.